

AI Knowledge Base

A Comprehensive Reference for RAG-Powered Chatbots

Topics: LLMs | RAG | Transformers | Prompt Engineering | Vector Databases | AI Agents

Large Language Models (LLMs)

Large Language Models (LLMs) are deep learning models trained on massive text corpora using the Transformer architecture. They learn statistical patterns in language and can generate coherent, contextually relevant text. Examples include GPT-4, Gemini, Claude, and LLaMA. LLMs are pre-trained on internet-scale data and fine-tuned for specific tasks.

Key properties of LLMs:

- Emergent capabilities: abilities that appear only at large scale (e.g., chain-of-thought reasoning).
- In-context learning: the model adapts to a task purely from examples in the prompt, without weight updates.
- Hallucination: LLMs can confidently produce factually incorrect outputs, making retrieval augmentation important.
- Tokenization: text is split into subword tokens; GPT-4 uses ~100k vocabulary BPE tokens.

Training pipeline: pre-training (next-token prediction on raw text) -> supervised fine-tuning (SFT) -> reinforcement learning from human feedback (RLHF) or direct preference optimization (DPO). Inference is auto-regressive: one token is sampled at a time conditioned on all previous tokens.

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) combines a neural retriever with a generative LLM. Instead of relying solely on parametric knowledge baked into model weights, RAG fetches relevant documents at query time and feeds them into the LLM context window.

RAG Pipeline Steps:

1. Ingestion: Documents are chunked, embedded, and stored in a vector database.
2. Retrieval: The user query is embedded; top-k similar chunks are retrieved via ANN search.
3. Augmentation: Retrieved chunks are prepended to the LLM prompt as context.
4. Generation: The LLM generates an answer grounded in the retrieved context.

Benefits of RAG:

- Reduces hallucination by grounding answers in real documents.
- Enables real-time knowledge updates without model retraining.
- Provides source attribution (cite which documents were used).
- Cost-effective compared to fine-tuning for knowledge injection.

Advanced RAG techniques include HyDE (Hypothetical Document Embeddings), query rewriting, re-ranking with cross-encoders, and multi-hop retrieval chains.

Transformer Architecture

The Transformer, introduced in 'Attention Is All You Need' (Vaswani et al., 2017), replaced RNNs for sequence modelling. Its core mechanism is multi-head self-attention, which allows every token to attend to every other token in $O(n^2)$ time.

Key components:

- Positional Encoding: Since Transformers lack recurrence, sinusoidal or learned positional embeddings inject sequence order information.
- Multi-Head Self-Attention: Q, K, V projections are computed in parallel across h heads, enabling the model to attend to different representation subspaces simultaneously.
- Feed-Forward Network (FFN): Two linear layers with a non-linearity (GELU/ReLU) applied position-wise after the attention sublayer.
- Layer Normalization and Residual Connections: Stabilize training in deep networks.

Variants:

- Encoder-only (BERT): masked language modelling; used for classification and embeddings.
- Decoder-only (GPT): causal language modelling; used for generation.
- Encoder-Decoder (T5, BART): sequence-to-sequence tasks like translation and summarization.

Scaling laws (Hoffmann et al., Chinchilla 2022) show optimal compute allocation requires scaling data and parameters proportionally.

Prompt Engineering

Prompt Engineering is the practice of crafting inputs to LLMs to elicit desired outputs without modifying model weights. Effective prompting can dramatically improve accuracy on reasoning, coding, and knowledge tasks.

Core techniques:

- Zero-shot prompting: Ask the model directly without examples.
- Few-shot prompting: Provide 2-8 input-output demonstrations before the query.
- Chain-of-Thought (CoT): Instruct the model to 'think step by step', enabling multi-step reasoning.
- Self-Consistency: Sample multiple CoT paths and majority-vote the final answer.
- Tree-of-Thoughts (ToT): Explore multiple reasoning branches and backtrack.
- ReAct: Interleave reasoning (Thought) with actions (tool calls) and observations.
- Role prompting: Assign a persona ('You are an expert data scientist...').

System prompt best practices:

- Be explicit about output format (JSON, markdown, bullet points).
- Specify constraints (word count, language, tone).
- Separate instructions from content using delimiters (triple backticks, XML tags).
- Instruct the model to cite sources when using RAG.

Prompt injection attacks (malicious instructions embedded in retrieved documents) are an important security concern in RAG systems.

Vector Databases

Vector databases store high-dimensional embedding vectors and enable fast approximate nearest-neighbour (ANN) search. They are the storage backbone of RAG systems.

How embeddings work:

Text is passed through an embedding model (e.g., Gemini embedding-001, OpenAI text-embedding-3) to produce a dense floating-point vector (e.g., 768 or 1536 dimensions). Semantically similar texts cluster close together in this embedding space as measured by cosine similarity or dot product.

Popular vector databases:

- ChromaDB: Open-source, embedded Python library, great for prototyping. Supports persistent disk storage and in-memory modes.
- Pinecone: Managed cloud service, scalable to billions of vectors.
- Weaviate: Open-source with hybrid BM25 + vector search.
- Qdrant: High-performance Rust-based vector DB with payload filtering.
- FAISS: Facebook's library for CPU/GPU ANN search; not a full database.

ANN algorithms used:

- HNSW (Hierarchical Navigable Small World): graph-based, high recall, used by ChromaDB/Weaviate.
- IVF (Inverted File Index): clustering-based, used by FAISS.
- Product Quantization (PQ): compresses vectors for memory efficiency.

Metadata filtering allows pre/post-filtering of results by attributes (date, source, category) before or after the ANN search.

AI Agents

AI Agents are LLM-powered systems that autonomously plan and execute multi-step tasks by calling tools, browsing the web, writing code, and coordinating with other agents.

Agent architectures:

- ReAct (Reason + Act): The model alternates between reasoning steps and tool invocations, observing results until the task is complete.
- Plan-and-Execute: A planner LLM generates a high-level task list; executor agents handle individual subtasks.
- Multi-Agent Systems: Orchestrator delegates subtasks to specialised sub-agents (researcher, coder, critic).

Common tools agents use:

- Web search (Google Search API, Bing)
- Code interpreter (Python sandbox execution)
- File system read/write
- RAG retrieval over a knowledge base
- API calls (calendar, email, databases)

Frameworks: LangChain, LangGraph, AutoGen, CrewAI, and Google's Agent Development Kit (ADK).

Key challenges:

- Long-horizon planning: maintaining coherent goals across many steps.
- Tool reliability: gracefully handling tool errors and unexpected outputs.
- Memory: episodic (conversation history), semantic (knowledge base), and procedural memory.
- Safety: avoiding harmful actions in open-ended agentic loops.